

Projeto de segmentação de imagens digitais de rocha

CPE-883 Tópicos Especiais Machine Learning

Vivian de Carvalho Rodrigues

DRE:125228569

15 de setembro de 2025

Table of Contents

1 Motivação

- ▶ **Motivação**
- ▶ Revisão bibliográfica
- ▶ Levantamento de dados e pré-processamento
- ▶ Método proposto
- ▶ Resultados
- ▶ Conclusões
- ▶ Referências Bibliográficas

Imagens digitais de rochas no setor O&G

1 Motivação

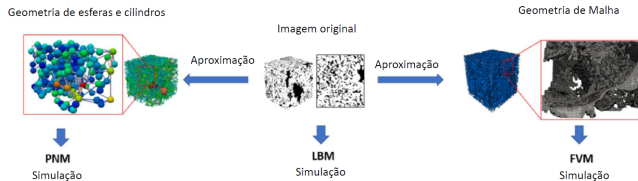


Figura: Simulações numéricas a partir de imagem digital de rochas-[1]

- A visualização da estrutura interna e opaca do meio poroso em 3D.
- Imagens de microtomografia (raio-X) que são segmentadas entre vazios e grãos.
- Determinar a petrofísica das rochas, como por exemplo a permeabilidade, que é essencial para alcançar a taxa comercial de produção do reservatório

Segmentação de imagens digitais

1 Motivação



Figura: Segmentação de imagens digitais

- Conversão de uma imagem em um conjunto de regiões de pixels representadas por uma máscara (rótulo).
- Pode ser por *descontinuidade* (detecção de bordas) ou *similaridade*, particionamento de uma imagem em regiões semelhantes (limiar)
- Objetivo final é extrair atributos da imagem (área, circularidade, quantidade de elementos, distância euclidiana e etc.)

Segmentação de imagens digitais de rochas

1 Motivação

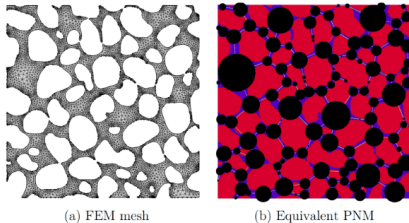


Figura: Segmentação de imagens digitais de rocha [2]

- **Objetivo:** realizar ensaios não destrutivos de amostras de rocha
 - Mapear a distribuição de poros da rocha
 - Quantificar o volume de vazios (poros conectados ou não)
 - Determinar a petrofísica das rochas analisadas

Table of Contents

2 Revisão bibliográfica

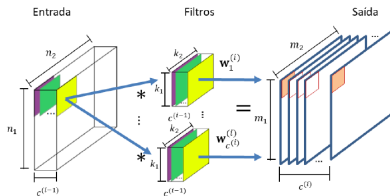
- ▶ Motivação
- ▶ **Revisão bibliográfica**
- ▶ Levantamento de dados e pré-processamento
- ▶ Método proposto
- ▶ Resultados
- ▶ Conclusões
- ▶ Referências Bibliográficas

Redes neurais convolucionais

2 Revisão bibliográfica

- Uma imagem digital é uma matriz de *pixels*
 - O operador convolucional é linear [3].
 - Processamento de um conjunto de características em paralelo, a partir de uma mesma entrada
 - Resultado da convolução com a entrada é calculado de maneira análoga a um núcleo de processamento de uma rede neural.

$$U_j^{(i)} = H_j^{(i)} (1^T b_j^{(i)} + U_j^{(i-1)} W_j^{(i)})$$



Redes-Kolmogorov-Arnold-KAN e CKAN

2 Revisão bibliográfica

- Descrição geral:
 - Redes KAN são inspiradas no teorema de Kolmogorov-Arnold para representação de funções multivariadas [5].
 - Diferentemente das MLPs tradicionais, cada **aresta** da rede possui uma **função univariada aprendível** (tipicamente um spline), no lugar de um peso fixo.
 - A versão convolucional (Convolutional KAN) estende o conceito para convoluções, onde cada elemento do kernel é uma função não-linear aprendida [6].

Formulação:

- Em KAN: $w_{ij} \rightarrow \phi_{ij}(x)$, onde ϕ_{ij} é um spline aprendível.
- Em CKAN (versão convolucional):

$$(I * K)_{i,j} = \sum_{k=1}^N \sum_{l=1}^M \phi_{kl}(a_{i+k,j+l})$$

, ϕ_{kl} são funções univariadas aprendíveis para cada posição no kernel

CapsNet — Matrix Capsules com EM Routing

2 Revisão bibliográfica

De acordo com [7]:

- Cada cápsula representa uma entidade com uma *matriz de pose* $\mathbf{M}_i$ e uma ativação escalar a_i .
- Cápsulas da camada inferior enviam **votos** para as superiores:

$$\mathbf{V}_{ij} = \mathbf{M}_i \cdot \mathbf{W}_{ij}$$

- O roteamento é feito via **EM (Expectation-Maximization)**, que ajusta a média, variância e ativação das cápsulas superiores.

CapsNet — Matrix Capsules com EM Routing

2 Revisão bibliográfica

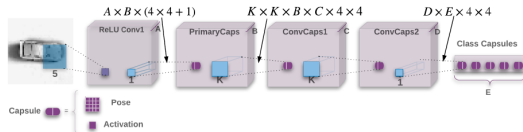


Figura: Arquitetura geral do modelo [7]

EM Routing [7]:

$$r_{ij} = \frac{a_j \cdot \mathcal{N}(\mathbf{V}_{ij} \mid \boldsymbol{\mu}_j, \boldsymbol{\sigma}_j^2)}{\sum_k a_k \cdot \mathcal{N}(\mathbf{V}_{ik} \mid \boldsymbol{\mu}_k, \boldsymbol{\sigma}_k^2)}$$

$$\boldsymbol{\mu}_j = \frac{\sum_i r_{ij} \cdot \mathbf{V}_{ij}}{\sum_i r_{ij}}, \quad \boldsymbol{\sigma}_j^2 = \frac{\sum_i r_{ij} \cdot (\mathbf{V}_{ij} - \boldsymbol{\mu}_j)^2}{\sum_i r_{ij}}$$

$$a_j = \text{activation} \left(\text{cost}(\boldsymbol{\mu}_j, \boldsymbol{\sigma}_j^2) \right)$$

SegCapsNet — Capsule Network para Segmentação

2 Revisão bibliográfica

Arquitetura básica: [8]

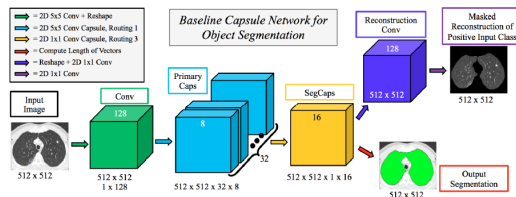


Figura: Arquitetura geral do modelo [7] [9]

- **Primary Capsules [9]:** Convolução seguida de *reshaping* em cápsulas vetoriais:

$$\mathbf{u}_i = \text{squash}(\mathbf{s}_i) = \frac{\|\mathbf{s}_i\|^2}{1 + \|\mathbf{s}_i\|^2} \frac{\mathbf{s}_i}{\|\mathbf{s}_i\|}$$

onde $\mathbf{s}_i$ é o vetor de ativação da cápsula i .

Arquitetura básica: [8]

- **Roteamento local:** Para cada cápsula da camada inferior i e superior j :

$$\hat{\mathbf{u}}_{j|i} = \mathbf{W}_{ij} \mathbf{u}_i$$

$\mathbf{W}_{ij}$ é a matriz de transformação e seguida de roteamento dinâmico baseado em coeficientes c_{ij} .

- **Deconvolutional Capsules:** recupera a resolução espacial e gera máscaras segmentadas.
- **Reconstrução da máscara:** para preservar a informação espacial da entrada.

SegCapsNet vs Matrix Capsules with EM Routing

2 Revisão bibliográfica

Diferenças fundamentais:

Matrix Capsules with EM Routing [7]

- Utiliza **matrizes de pose** $\mathbf{M}_i \in \mathbb{R}^{4 \times 4}$.
- Roteamento por **Expectation-Maximization (EM)** com estimativas de média e variância Gaussiana.
- Geração de votos via multiplicação matricial e agrupamento probabilístico.
- Projetada para **reconhecimento e modelagem hierárquica de pose**.

SegCapsNet [8]

- Utiliza **cápsulas vetoriais**.
- **Roteamento dinâmico local** baseado em coeficientes de afinidade c_{ij} .
- Aplicação de transformações $\mathbf{W}_{ij}$ para votos $\hat{\mathbf{u}}_{j|i}$.
- Otimizada para **segmentação**, com preservação espacial e menor complexidade computacional.

DeepONet — Aprendizado de Operadores

2 Revisão bibliográfica

Definição [10]: Rede que aprende um operador contínuo

$$\mathcal{G} : u(\cdot) \mapsto \mathcal{G}(u)(\cdot)$$

Onde u pertence a um espaço de funções de entrada, e $\mathcal{G}(u)$ é função de saída.

Arquitetura Branch & Trunk:

- *Branch network:* $b(u) = (b_1, \dots, b_p)$.
- *Trunk network:* recebe ponto de avaliação y , produz características $t(y) = (t_1(y), \dots, t_p(y))$.
- Combinação para estimar saída:

$$\mathcal{G}_\theta(u)(y) \approx \sum_{i=1}^p b_i(u) t_i(y) + b_0$$

Função de custo:

$$\mathcal{L} = \mathbb{E}_{u,y} \|\mathcal{G}_\theta(u)(y) - \mathcal{G}(u)(y)\|^2$$

Convolutional DeepONet:

2 Revisão bibliográfica

Modelo DeepONet convolucional [11]:

- *Branch net 1 (CNN)* processa máscara anatômica $\rightarrow$ representação $g_{\text{geo}}(\mathbf{u})$ em $\mathbb{R}^q$.
- *Branch net 2 (FCNN)* processa localização do transdutor $\rightarrow$ representação $g_{\text{src}}(\mathbf{z})$ em $\mathbb{R}^r$.
- *Trunk net (FCNN)* recebe os pontos espaciais $\mathbf{x}$, produz $t(\mathbf{x}) \in \mathbb{R}^p$.
- Representação combinada:

$$p(\mathbf{x}) = \mathcal{G}_{\theta}(\mathbf{u}, \mathbf{z})(\mathbf{x}) \approx \sum_{i=1}^p b_i(\mathbf{u}, \mathbf{z}) t_i(\mathbf{x}) + b_0$$

onde $b_i(\mathbf{u}, \mathbf{z})$ concatena ou combina as representações dos dois branch nets.

Loss e métricas:

$$\mathcal{L}_{\text{rel-L}_2} = \mathbb{E}_{\mathbf{u}, \mathbf{z}, \mathbf{x}} \frac{\|p_{\text{pred}}(\mathbf{x}) - p_{\text{sim}}(\mathbf{x})\|^2}{\|p_{\text{sim}}(\mathbf{x})\|^2}$$

Table of Contents

3 Levantamento de dados e pré-processamento

- ▶ Motivação
- ▶ Revisão bibliográfica
- ▶ **Levantamento de dados e pré-processamento**
- ▶ Método proposto
- ▶ Resultados
- ▶ Conclusões
- ▶ Referências Bibliográficas

Dataset

3 Levantamento de dados e pré-processamento

- 4 amostras de micro tomografia
- Total de imagens de entrada: 5558 (8.5 Gb em disco)
- Total de máscaras: 5558 (4.3 Gb em disco)
- Resolução das imagens brutas de entrada 40 microns
- Marcação dos rótulos *fundo*, *rocha* e *poro* através do programa *Comet Tech Dragonfly* ([12])

Máscaras multiescalas (pré-tratamento)

3 Levantamento de dados e pré-processamento

Github: [13]

```
# Função para extrair o número do slice do nome do arquivo (ex: "rocha0001.tiff" -> "0001")
def extrair_numero(nome_arquivo):
    m = re.search(r'\d+', nome_arquivo)
    return m.group(0) if m else None

# Listar todos arquivos da pasta que começam com 'rocha' e terminam com '.tiff'
arquivos_rocha = [f for f in os.listdir(pasta) if f.startswith('rocha') and f.endswith('.tiff')]

print(f"{len(arquivos_rocha)} arquivos de rocha encontrados.")

for arquivo_rocha in sorted(arquivos_rocha):
    numero = extrair_numero(arquivo_rocha)
    if numero is None:
        print(f"Não consegui extrair número do arquivo {arquivo_rocha}, pulando...")
        continue

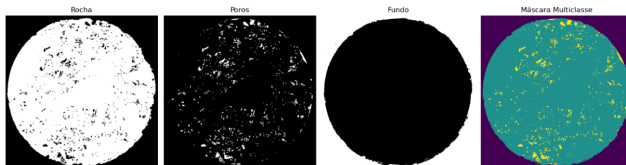
    arquivo_poros = f"poro{numero}.tiff"
    arquivo_fundo = f"fundo{numero}.tiff"
    saida = f"mask{numero}.tiff"
```

```
# Carregar imagens
rocha = iio.imread(caminho_rocha)
poros = iio.imread(caminho_poros)
fundo = iio.imread(caminho_fundo)

# Converter 255 -> 1
rocha_bin = (rocha > 128).astype(np.uint8)
poros_bin = (poros > 128).astype(np.uint8)
fundo_bin = (fundo > 128).astype(np.uint8)

# Criar máscara multiclasse
mask = np.zeros_like(rocha_bin, dtype=np.uint8)
mask[fundo_bin == 1] = 0
mask[rocha_bin == 1] = 1
mask[poros_bin == 1] = 2

# Salvar máscara
iio.imwrite(caminho_saida, mask)
print(f"Máscara salva para slice {numero} em {caminho_saida}")
```



Dataloader

3 Levantamento de dados e pré-processamento

Github: [13]

- Percorre todas as subpastas (DatasetA, B, ...), procurando por inputs/ e masks/.
- Leitura normalização da imagem + leitura de máscara
- Redimensionamento (512 x 512 pixels)
- Conversão para tensores

```
dataset/
├── DatasetA/
│   ├── inputs/
│   └── masks/
└── DatasetB/
    ├── inputs/
    └── masks/
```

Dataloader

3 Levantamento de dados e pré-processamento

Github: [13]

```
def __getitem__(self, idx):
    input_path, mask_path, amostra_nome = self.samples[idx]
    filename = os.path.basename(input_path) # ex: "sampleA0001.tiff"

    # Leitura das imagens
    image = iio.imread(input_path).astype(np.float32) / 255.0
    mask = iio.imread(mask_path).astype(np.int64)

    # Redimensiona imagem e máscara
    image = resize(
        image, self.output_shape,
        preserve_range=True, anti_aliasing=True
    )
    mask = resize(
        mask, self.output_shape,
        order=0, preserve_range=True, anti_aliasing=False
    ).astype(np.int64)

    # Adiciona canal à imagem
    image = np.expand_dims(image, axis=0) # [1, H, W]

    if self.transform:
        image, mask = self.transform(image, mask)

    return (
```

```
# Split treino, validação e teste
def split_dataset(dataset, train_ratio=0.7, val_ratio=0.15, test_ratio=0.15, s
    assert abs(train_ratio + val_ratio + test_ratio - 1.0) < 1e-6, "Soma das p

    total_size = len(dataset)
    train_size = int(train_ratio * total_size)
    val_size = int(val_ratio * total_size)
    test_size = total_size - train_size - val_size

    return random_split(
        dataset,
        [train_size, val_size, test_size],
        generator=torch.Generator().manual_seed(seed)
    )
```

Table of Contents

4 Método proposto

- ▶ Motivação
- ▶ Revisão bibliográfica
- ▶ Levantamento de dados e pré-processamento
- ▶ **Método proposto**
- ▶ Resultados
- ▶ Conclusões
- ▶ Referências Bibliográficas

Projeto - estrutura

4 Método proposto

Github: [14]

```
Projeto_segmentacao/
├── rock_seg_loader/
│   └── rock_dataset_multi.py
├── rock_seg_model/
│   ├── caps_rock_seg/
│   │   ├── model_caps.py
│   │   ├── caps_config.py
│   │   └── __init__.py
│   ├── ckan_rock_seg/
│   │   ├── model_ckan.py
│   │   ├── kan_conv.py
│   │   ├── convolution.py
│   │   ├── ckan_config.py
│   │   └── __init__.py
│   ├── deeponet_rock_seg/
│   │   ├── model_deeponet.py
│   │   ├── deeponet_config.py
│   │   └── __init__.py
│   ├── cnn_rock_seg/
│   │   ├── model_cnn.py
│   │   ├── cnn_config.py
│   │   └── __init__.py
│   └── __init__.py
├── metrics.py
├── main_ckan.py
├── main_caps.py
├── main_cnn.py
├── main_deeponet.py
└── results /
```

```
results_/
├── CapsNet/
│   ├── val_set/      => Epoch_1 /Epoch_2 (....) Predições de validação
│   ├── test_set/     => Predições de teste
│   ├── loss_curve.png => Curvas loss do conjunto de treino ('train_loss') , validação ('
│   ├── DICE_IOU_curve.png => Curvas Dice /IOU score que já estão sendo impressas no terminal
│   └── results.txt    => Escrever os dados para que possa ser armazenado em tabela
                        época | train_loss | val_loss | test_loss |
                        val Dice_score | val IOU_score | test Dice_score | test IOU_score
                        (dados impressos no terminal)
├── CKAN/
│   ├── val_set/
│   ├── test_set/
│   ├── loss_curve.png
│   ├── DICE_IOU_curve.png
│   └── results.txt
├── deeponet/
│   ├── val_set/
│   ├── test_set/
│   ├── loss_curve.png
│   ├── DICE_IOU_curve.png
│   └── results.txt
├── cnn/
│   ├── val_set/
│   ├── test_set/
│   ├── loss_curve.png
│   ├── DICE_IOU_curve.png
│   └── results.txt
```

Comparação entre Modelos de Segmentação

4 Método proposto

Item	CNN	CKAN	CapsNet (SegCapsNet)	DeepONetConv
Arquitetura	CNN padrão com 3 camadas convolucionais	KAN convolucional (splines)	CapsNet adaptada para segmentação	DeepONet convolucional (Branch + Trunk)
Entrada	Imagem cinza (1 canal)	Imagem cinza (1 canal)	Imagem cinza (1 canal)	Imagem cinza (1 canal)
Nº classes	3 (fundo, rocha, poro)	3 (fundo, rocha, poro)	3 (fundo, rocha, poro)	3 (fundo, rocha, poro)
Camadas principais	3 × Conv2D + camada final (Conv2D kernel=1)	3 × KANConv2D + camada final Conv2D	Conv2D → PrimaryCaps → Deconv → Conv final	Branch Net (Conv2D) + Trunk Net (Conv2D) + FC
Ativação	ReLU	Splines aprendíveis (KAN)	Squash (cápsulas)	ReLU
Fusão de features	Convolução tradicional	Convoluções KAN	Cápsulas (caps_dim)	Produto escalar (branch × trunk)
Hidden Dim	–	–	–	64
Função de perda	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss	CrossEntropyLoss
Otimizador	Adam	Adam	Adam	Adam
Resolução saída	128 × 128	64 × 64	128 × 128	128 × 128
Métricas	Dice Score, IoU	Dice Score, IoU	Dice Score, IoU	Dice Score, IoU

Projeto - Arquitetura CNN

4 Método proposto

Github: [14]

- **Input:** $128 \times 128 \times 1$
 - Conv2D ($1 \rightarrow 4$): camada convolucional padrão (ex: kernel 3×3 , stride 1, padding 1)
 - ReLU: função de ativação
 - Conv2D ($4 \rightarrow 8$): camada convolucional padrão (kernel 3×3 , stride 1, padding 1)
 - ReLU: função de ativação
 - Conv2D ($8 \rightarrow 16$): camada convolucional padrão (kernel 3×3 , stride 1, padding 1)
 - ReLU: função de ativação
 - Conv2D ($16 \rightarrow 3$, kernel=1): camada final para gerar logits por classe (sem ativação)
- **Output:** $128 \times 128 \times 3$

```
class CNNSegmentationModel(nn.Module):
    def __init__(self, config=CNN_CONFIG):
        super().__init__()
        ch = config["channels"]
        nc = config["num_classes"]

        # 3 camadas convolucionais padrão
        self.conv1 = nn.Conv2d(
            config["img_channels"], ch[0],
            kernel_size=config["kernel_size"],
            stride=config["stride"],
            padding=config["padding"]
        )
        self.conv2 = nn.Conv2d(
            ch[0], ch[1],
            kernel_size=config["kernel_size"],
            stride=config["stride"],
            padding=config["padding"]
        )
        self.conv3 = nn.Conv2d(
            ch[1], ch[2],
            kernel_size=config["kernel_size"],
            stride=config["stride"],
            padding=config["padding"]
        )
```


Projeto - Arquitetura CKAN

4 Método proposto

Github: [14]

- **Input:** $128 \times 128 \times 1$

- CKANConv2DReal ($1 \rightarrow 4$): KAN Convolutional Layer (3×3), ativação KAN (ex: 16 splines por canal)
- Identity: Sem downsampling
- CKANConv2DReal ($4 \rightarrow 8$): KAN Convolutional Layer (3×3), ativação KAN (ex: 32 splines por canal)
- Identity: Sem downsampling
- CKANConv2DReal ($8 \rightarrow 16$): KAN Convolutional Layer (3×3), ativação KAN (ex: 64 splines por canal)
- Identity: Sem downsampling
- Conv2D ($16 \rightarrow 3$, kernel=1): Camada final (gera logits por classe, sem ativação)

- **Output:** $64 \times 64 \times 3$

```
class CKANSegmentationModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        ch = config["channels"]
        ks = config["kernel_size"]
        st = config["stride"]
        pad = config["padding"]
        dil = config["dilation"]
        nc = config["num_classes"]

        self.ckan1 = CKANConv2DReal(ch[0], ch[1], ks, st, pad, dil)
        self.pool1 = nn.Identity()

        self.ckan2 = CKANConv2DReal(ch[1], ch[2], ks, st, pad, dil)
        self.pool2 = nn.Identity()

        self.ckan3 = CKANConv2DReal(ch[2], ch[3], ks, st, pad, dil)
        self.pool3 = nn.Identity()

        self.final_conv = nn.Conv2d(ch[3], nc, kernel_size=1)

    def forward(self, x):
        x = self.pool1(self.ckan1(x))
```

Projeto - Arquitetura CapsNet

4 Método proposto

Github: [14]

- Conv2d
 - Extrai features iniciais da imagem
 - Saída: [B, 32, 128, 128]
- PrimaryCaps
 - Projeta os features em cápsulas vetoriais (grupos de neurônios)
 - Saída: [B, caps channels=8, caps dim=8, H, W]
- Squash (ativação)
 - Normaliza os vetores das cápsulas
 - Mantém direção (informação) e comprime a magnitude para [0, 1] (confiança)
- Reorganização (view)
 - Achata a dimensão vetorial: [B, 64, H, W]
 - Prepara para convolução tradicional

```
class SegCapsNet(nn.Module):
    def __init__(self, img_channels=1, num_classes=3):
        super().__init__()
        self.conv1 = nn.Conv2d(img_channels, 32, kernel_size=5, stride=1, padding=2)
        self.primary_caps = PrimaryCaps(32, caps_channels=8, caps_dim=8, kernel_size=1, stride=1)
        # stride=2 removido para não dobrar tamanho
        self.deconv = nn.ConvTranspose2d(8*8, 32, kernel_size=1, stride=1)
        self.final = nn.Conv2d(32, num_classes, kernel_size=1)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        caps = self.primary_caps(x)
        caps = caps.view(x.size(0), -1, caps.size(3), caps.size(4)) # [B, C*dim, H, W]
        x = F.relu(self.deconv(caps))
        out = self.final(x)
        return out
```

Projeto - Arquitetura CapsNet

4 Método proposto

Github: [14]

- ConvTranspose2d
 - Refina os mapas e mantém resolução (sem upsampling neste caso)
 - Saída: [B, 32, 128, 128]
- Conv2d final
 - Gera os logits de segmentação
 - Saída: [B, 3, 128, 128] (3 classes por pixel)
- Saída final
 - Tensor de *logits* com shape [B, 3, 128, 128] — um mapa para cada classe e pixel

```
class PrimaryCaps(nn.Module):
    def __init__(self, in_channels, caps_channels, caps_dim, kernel_size, stride):
        super().__init__()
        self.caps_dim = caps_dim
        self.caps_channels = caps_channels
        self.conv = nn.Conv2d(in_channels, caps_channels * caps_dim, kernel_size, stride)

    def forward(self, x):
        batch = x.size(0)
        out = self.conv(x)
        out = out.view(batch, self.caps_channels, self.caps_dim, out.size(2), out.size(3))
        out = self.squash(out)
        return out

    def squash(self, s):
        norm = torch.norm(s, dim=2, keepdim=True)
        return (norm**2 / (1 + norm**2)) * (s / (norm + 1e-8))
```

Projeto - Arquitetura DeepOnet

4 Método proposto

Github: [14]

- Aplica a lógica Branch/Trunk do DeepONet com convoluções
- O *branch* net pode receber como entrada imagens
- O *trunk* net recebe coordenadas ou informações espaciais.

```
class DeepOnetConv(nn.Module):
    def __init__(self, config):
        super().__init__()
        # Branch net - processa a entrada
        layers_branch = []
        in_ch = config["img_channels"]
        for out_ch, k, s in zip(
            config["branch_conv_channels"],
            config["branch_kernel_sizes"],
            config["branch_strides"]
        ):
            layers_branch += [
                nn.Conv2d(in_ch, out_ch, kernel_size=k, stride=s, padding=k // 2),
                nn.ReLU(inplace=True)
            ]
            in_ch = out_ch
        self.branch_net = nn.Sequential(*layers_branch)

        # Trunk net - também pode processar informação da entrada ou outra fonte
        layers_trunk = []
        in_ch = config["img_channels"]
        for out_ch, k, s in zip(
            config["trunk_conv_channels"],
            config["trunk_kernel_sizes"],
            config["trunk_strides"]
        ):
            layers_trunk += [
                nn.Conv2d(in_ch, out_ch, kernel_size=k, stride=s, padding=k // 2),
                nn.ReLU(inplace=True)
            ]
            in_ch = out_ch
        self.trunk_net = nn.Sequential(*layers_trunk)
```

Projeto - Arquitetura DeepOnet

4 Método proposto

Github: [14]

- Após a multiplicação $branch \times trunk$, o resultado é “achatado” (*flatten*) e passado por uma camada densa com hidden dim neurônios, com ReLU.
- Essa camada representa uma combinação intermediária das informações extraídas pelo *branch* e pelo *trunk*, antes da projeção para a saída final de segmentação.

```
# Combinação via operador (produto escalar)
hidden = config["hidden_dim"]
self.fc = nn.Sequential(
    nn.Flatten(),
    nn.Linear(in_ch * config["output_shape"][0] * config["output_shape"][1], hidden),
    nn.ReLU(inplace=True),
    nn.Linear(hidden, config["num_classes"] * config["output_shape"][0] * config["output_shape"][1])
)
self.config = config

def forward(self, x):
    b = self.branch_net(x)
    t = self.trunk_net(x)
    # junção simples - produto escalar por canal
    h = (b * t).flatten(1)
    out = self.fc(h)
    B, C, H, W = b.size(0), self.config["num_classes"], *self.config["output_shape"]
    return out.view(B, C, H, W)
```

Convoluções na Trunk Net (DeepONet adaptado)

4 Método proposto

- O modelo original trabalha com mapeamentos funcionais (ex: PDEs), onde a trunk processa coordenadas.
- Para segmentar imagens, usar convoluções na trunk permite extrair **padrões locais** relevantes (bordas, texturas, regiões).
- **combinação via produto escalar** ocorre entre tensores com alinhamento espacial mais coerente.
- **Adaptação moderna [15]:** Trabalhos recentes adaptam DeepONet com CNNs em trunk/branch para tarefas visuais (ex: reconstrução de campos, super-resolução, segmentação).

Métricas de Avaliação - Dice Score

4 Método proposto

Github: [14]

- Mede a sobreposição entre a predição e a máscara real.

- Fórmula:

$$Dice = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

- Varia entre 0 (sem sobreposição) e 1 (sobreposição perfeita).
- Calculado por classe e depois feita a média.

```
def dice_score(pred, target, num_classes):
    dice = []
    pred = pred.cpu().numpy()
    target = target.cpu().numpy()

    for c in range(num_classes):
        pred_c = (pred == c).astype(np.uint8)
        target_c = (target == c).astype(np.uint8)

        inter = (pred_c * target_c).sum()
        union = pred_c.sum() + target_c.sum()

        if union > 0:
            dice.append(2.0 * inter / union)

    return np.mean(dice)
```

Métricas de Avaliação - IoU Score

4 Método proposto

Github: [14]

- Mede a interseção sobre a união das regiões segmentadas.

- Fórmula:

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

- Também varia de 0 a 1 — valores maiores indicam melhor performance.
- Usado frequentemente em benchmarks de segmentação semântica.

```
def iou_score(pred, target, num_classes):
    ious = []
    pred = pred.cpu().numpy()
    target = target.cpu().numpy()

    for c in range(num_classes):
        pred_c = (pred == c).astype(np.uint8)
        target_c = (target == c).astype(np.uint8)

        inter = (pred_c * target_c).sum()
        union = pred_c.sum() + target_c.sum() - inter

        if union > 0:
            ious.append(inter / union)

    return np.mean(ious)
```

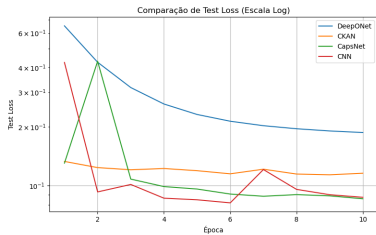
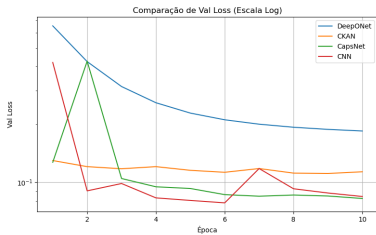
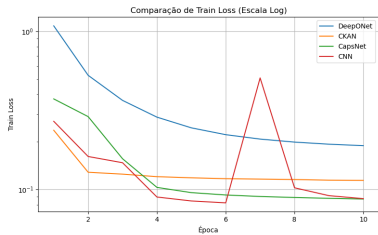

Table of Contents

5 Resultados

- ▶ Motivação
- ▶ Revisão bibliográfica
- ▶ Levantamento de dados e pré-processamento
- ▶ Método proposto
- ▶ **Resultados**
- ▶ Conclusões
- ▶ Referências Bibliográficas

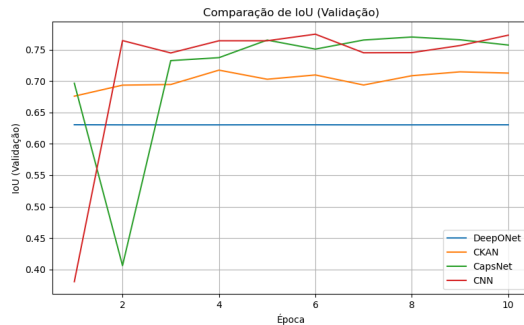
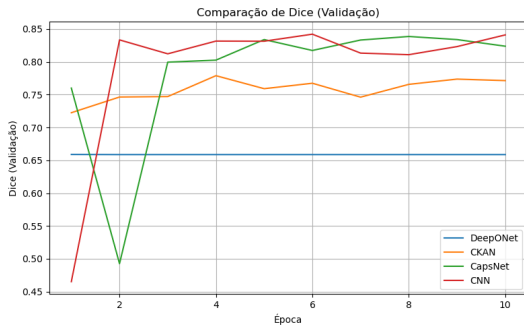
Loss

5 Resultados



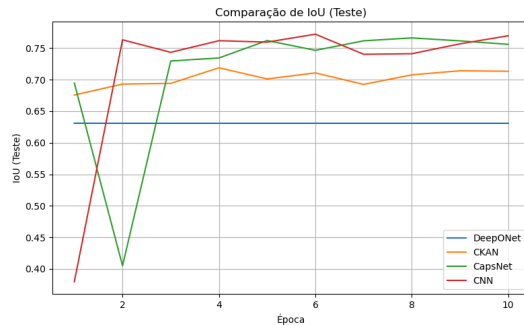
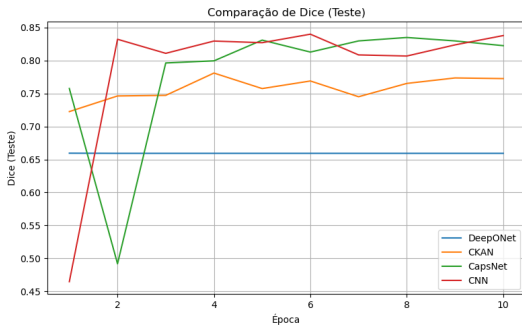
Dice e IoU

5 Resultados



Dice e IoU

5 Resultados



Matriz Confusão - CNN

5 Resultados

Conjunto de teste - época 10

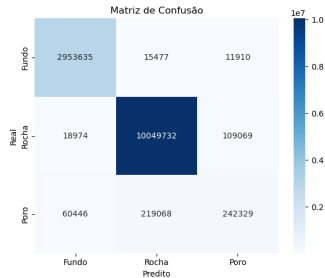
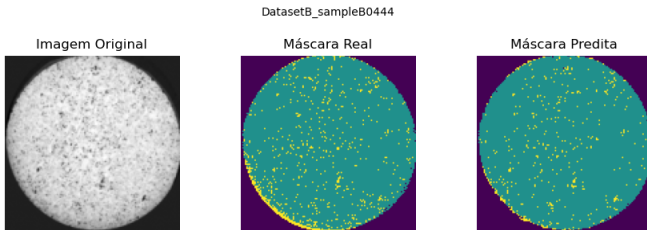


Figura: Conjunto de teste

Matriz Confusão - CKAN

5 Resultados

Conjunto de teste - época 10

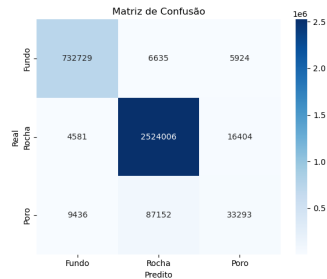
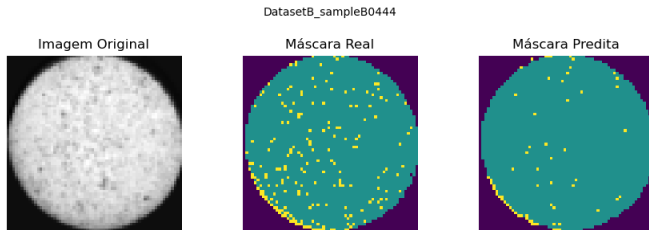


Figura: Conjunto de teste

Matriz Confusão - CapsNet

5 Resultados

Conjunto de teste - época 10

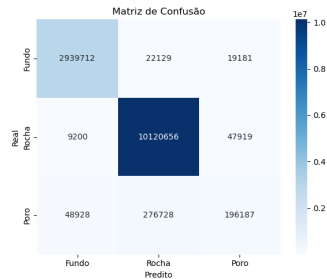
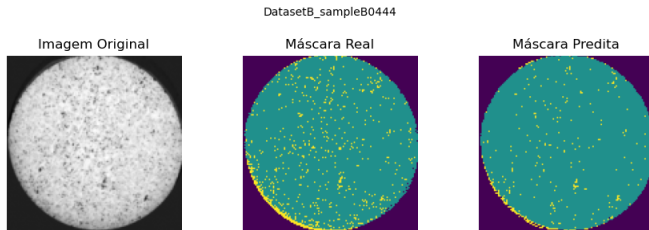


Figura: Conjunto de teste

Matriz Confusão - DeepONet

5 Resultados

Conjunto de teste - época 10

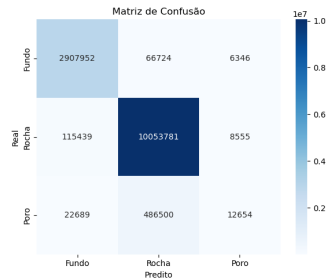
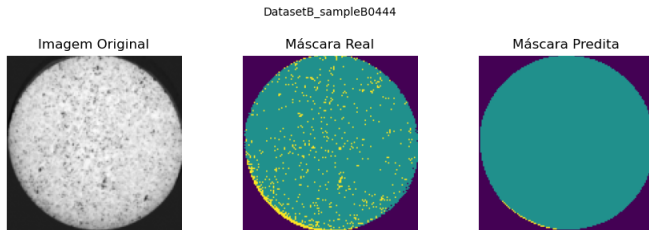


Figura: Conjunto de teste

Modelos selecionados para validação cruzada

5 Resultados

Github: [16]

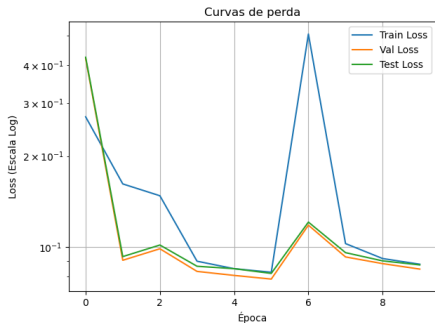


Figura: CNN

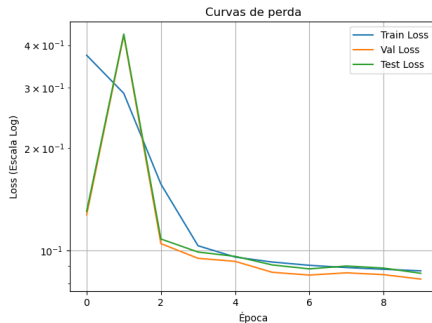


Figura: CapsNet

Resultados validação cruzada - 10 folds

5 Resultados

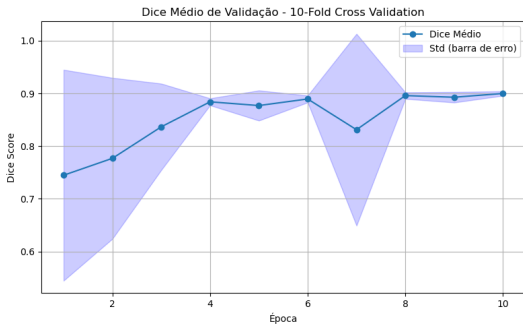


Figura: CNN

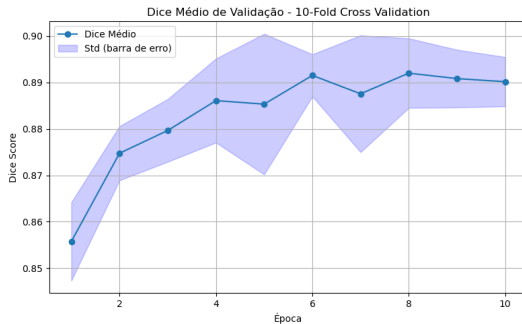


Figura: CapsNet

Table of Contents

6 Conclusões

- ▶ Motivação
- ▶ Revisão bibliográfica
- ▶ Levantamento de dados e pré-processamento
- ▶ Método proposto
- ▶ Resultados
- ▶ **Conclusões**
- ▶ Referências Bibliográficas

Conclusões

6 Conclusões

- Foram avaliados quatro modelos de segmentação aplicados a **imagens de tomografia de rochas: CNN, CapsNet, CKAN e DeepONet**.
- **CapsNet** e **CNN** apresentaram os melhores resultados em termos de:
 - Métricas de segmentação (Dice e IoU);
 - Desempenho em validação e teste;
 - Capacidade de generalização avaliada via **validação cruzada (10-fold)**.
- O modelo **CKAN** não é ruim, mas foi treinado com resolução menor devido a limitações computacionais (tempo maior de execução), o que pode ter impactado seu desempenho.
- **CapsNet** se destacou pela **estabilidade entre os folds**, demonstrando **maior robustez** mesmo com leve diferença no desempenho médio.
- **DeepONet** apresentou os piores resultados, mostrando-se inadequado para tarefas de segmentação de imagens nesse contexto.

Trabalhos futuros

6 Conclusões

- Implementar arquiteturas mais profundas e específicas (U-Net).
- Aplicar técnicas avançadas de data augmentation e funções de perda otimizadas para segmentação.
- Validar resultados com métricas petrofísicas, ampliando a confiabilidade prática.
- Testar em bases maiores e multiescalares para robustez e generalização.

Table of Contents

7 Referências Bibliográficas

- ▶ Motivação
- ▶ Revisão bibliográfica
- ▶ Levantamento de dados e pré-processamento
- ▶ Método proposto
- ▶ Resultados
- ▶ Conclusões
- ▶ **Referências Bibliográficas**

Referências Bibliográficas

7 Referências Bibliográficas

- [1] M. Tembely, A. M. AlSumaiti, and W. S. Alameri, "Machine and deep learning for estimating the permeability of complex carbonate rock from x-ray micro-computed tomography," *Energy Reports*, vol. 7, pp. 1460–1472, 2021.
- [2] J. Gostick, M. Aghighi, J. Hinebaugh, T. Tranter, M. A. Hoeh, H. Day, B. Spellacy, M. H. Sharqawy, A. Bazylak, A. Burns, W. Lehnert, and A. Putz, "Openpnm: A pore network modeling package," *Computing in Science & Engineering*, vol. 18, no. 4, pp. 60–74, 2016.
- [3] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [4] A. G. Evsukoff, *Inteligência Computacional: Fundamentos e aplicações*. Rio de Janeiro: E-papers, 1 ed., 2020.

Referências Bibliográficas

7 Referências Bibliográficas

- [5] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, “Kan: Kolmogorov-arnold networks,” 2025.
- [6] A. D. Bodner, A. S. Tepsich, J. N. Spolski, and S. Pourteau, “Convolutional kolmogorov-arnold networks,” 2025.
- [7] N. F. Geoffrey Hinton, Sara Sabour, “Matrix capsules with em routing,” 2018.
- [8] R. LaLonde and U. Bagci, “Capsules for object segmentation,” 2018.
- [9] S. Sabour, N. Frosst, and G. E. Hinton, “Dynamic routing between capsules,” 2017.
- [10] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via deeponet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, vol. 3, p. 218–229, Mar. 2021.

Referências Bibliográficas

7 Referências Bibliográficas

- [11] A. Kumar, X. Zhi, Z. Ahmad, M. Yin, and A. Manbachi, “Convolutional deep operator networks for learning nonlinear focused ultrasound wave propagation in heterogeneous spinal cord anatomy,” 2024.
- [12] Comet, “Dragonfly (commercial software).” Eletronic, 2015.
<https://dragonfly.comet.tech/>.
- [13] V. Rodrigues, “Dataloader.” Github, Sept. 2025.
- [14] V. Rodrigues, “Modelos.” Github, Sept. 2025.

Referências Bibliográficas

7 Referências Bibliográficas

- [15] J. He, S. Koric, S. Kushwaha, J. Park, D. Abueidda, and I. Jasiuk, “Novel deepo-net architecture to predict stresses in elastoplastic structures with variable complex geometries and loads,” *Computer Methods in Applied Mechanics and Engineering*, vol. 415, p. 116277, Oct. 2023.
- [16] V. Rodrigues, “Validação cruzada.” Github, Sept. 2025.